



GOVERNMENT OF INDIA • DEPARTMENT OF SPACE

Vikram Sarabhai Space Centre

Thiruvananthapuram – 695022, Kerala, India

INTERNSHIP COMPLETION REPORT

on

VISION

**Design and Development of an Offline Multi-Modal
Retrieval-Augmented Generation (RAG) Platform
for Intelligent Document Understanding**

Submitted by

Ms. Sreeharini J

B.Tech Computer Science & Engineering | 6th Semester

NSS College of Engineering, Palakkad

Internship carried out at

Vikram Sarabhai Space Centre (VSSC), ISRO

Human Resource Development Division

Ref No: HRDD/INT-P/0/26 • Duration: 03 June 2026 – 03 July 2026

Supervisor: Mr. Majji Sathwik Reddy, Scientist/Engineer SC, COM/CITG/MSA

CERTIFICATE



GOVERNMENT OF INDIA • DEPARTMENT OF SPACE
VIKRAM SARABHAI SPACE CENTRE

Thiruvananthapuram – 695022

This is to certify that the internship report titled "Vision: Design and Development of an Offline Multi-Modal Retrieval-Augmented Generation (RAG) Platform for Intelligent Document Understanding" is a bonafide record of the work carried out by Ms. Sreeharini J, a 6th-semester B.Tech (Computer Science & Engineering) student from NSS College of Engineering, Palakkad, during her internship at the Vikram Sarabhai Space Centre (VSSC), Thiruvananthapuram.

The internship was completed under my supervision during the period 03 June 2026 to 03 July 2026 (Ref No: HRDD/INT-P/0/26). The work presented is original and has not been submitted elsewhere for the award of any degree or diploma.

The intern demonstrated commendable understanding of advanced AI systems, containerized deployment architectures, and applied machine learning in a sensitive organizational context.

Signature of the Project Supervisor

Name: Mr. Majji Sathwik Reddy
Designation: Scientist/Engineer SC
Division: COM/CITG/MSA, VSSC

Approving Authority — HRDD

Date: _____
Official Seal: _____

DECLARATION

I, Ms. Sreeharini J, hereby declare that the internship report titled "Vision: Design and Development of an Offline Multi-Modal Retrieval-Augmented Generation (RAG) Platform for Intelligent Document Understanding" represents the original work carried out by me during my internship at the Vikram Sarabhai Space Centre (VSSC), Thiruvananthapuram, from 03 June 2026 to 03 July 2026.

I further declare that this work was completed under the guidance and supervision of Mr. Majji Sathwik Reddy, Scientist/Engineer SC, COM/CITG/MSA, VSSC, and that the contents of this report have not been submitted to any other university or institution for the award of any degree, diploma, or fellowship.

All sources of information used in this work have been duly acknowledged and referenced in the bibliography.

Place: Thiruvananthapuram Date: _____	Signature Sreeharini J · NSS College of Engineering
--	---

ACKNOWLEDGEMENT

I would like to express my profound gratitude to the Vikram Sarabhai Space Centre (VSSC) and the Human Resource Development Division (HRDD) for providing me with the extraordinary opportunity to intern at such a premier national research organization. Being a part of VSSC's ecosystem — even briefly — has been a transformative experience that has reshaped my understanding of systems-level AI engineering.

My deepest thanks go to my internship supervisor, Mr. Majji Sathwik Reddy (Scientist/Engineer SC, COM/CITG/MSA), for his continuous support, meticulous technical guidance, and patient mentorship throughout the duration of this project. His expertise in artificial intelligence and distributed systems engineering was instrumental in the successful conception, architecture, and offline deployment of this multi-modal platform.

I extend my sincere gratitude to the entire COM/CITG/MSA division team for their cooperative environment, technical discussions, and the invaluable insights they shared about real-world secure computing infrastructure.

I am deeply indebted to the Principal, Dr. Head of the Department (Computer Science & Engineering), and the esteemed faculty members of NSS College of Engineering, Palakkad, for equipping me with the foundational computer science and engineering principles required to undertake a project of this scale and complexity.

Finally, I would like to thank my parents, peers, and friends for their unwavering encouragement and support throughout this internship period.

Sreeharini J

ABSTRACT

Modern aerospace and defense organizations generate vast amounts of heterogeneous, highly sensitive data — ranging from classified mission text documents to complex tabular telemetry logs and scanned engineering diagrams. Retrieving precise, contextual information from these multi-modal silos securely, without relying on third-party cloud architectures, is an acute computational challenge and a strict operational necessity.

This report details the design, engineering, and deployment of Vision, an entirely containerized, offline-capable Multi-Modal Retrieval-Augmented Generation (RAG) platform purpose-built for secure enterprise environments. The architecture embraces a local-first, privacy-first paradigm, guaranteeing that no organizational intellectual property traverses the public internet. The system natively ingests heterogeneous documents using advanced Optical Character Recognition (PaddleOCR) and layout-aware structural parsing (Docling). To eliminate semantic hallucinations and relational blindness, ingested data is processed via sliding-window semantic chunking and stored simultaneously in a high-dimensional vector database (Qdrant) and a relational knowledge graph (Neo4j).

Powered by the state-of-the-art Qwen2.5-VL Vision-Language Models — served via the high-throughput vLLM engine using PagedAttention memory management — and orchestrated through a FastAPI backend with asynchronous Celery task queues, the platform delivers a robust conversational interface capable of reading text, interpreting embedded diagrams, and synthesizing highly accurate, citation-backed answers.

The result is a battle-hardened, air-gapped AI infrastructure capable of cold-booting natively on Red Hat Enterprise Linux (RHEL) GPU servers — guaranteeing zero data leakage, sub-second hybrid retrieval, and enterprise-grade multi-modal comprehension. This project sets a benchmark for sovereign, on-premise AI deployment in nationally critical institutions.

Keywords: Retrieval-Augmented Generation (RAG), Vision-Language Models, Knowledge Graph, Vector Database, Air-Gapped Deployment, Multi-Modal AI, PagedAttention, AWQ Quantization, Enterprise AI, VSSC, ISRO

TABLE OF CONTENTS

Preliminary Sections

Certificate..... i

Declaration..... ii

Acknowledgement..... iii

Abstract..... iv

List of Figures..... v

List of Tables..... v

Main Report

1. Introduction..... 1

 1.1 Background and Motivation.....1

 1.2 Scope of the Internship.....2

 1.3 Report Structure.....2

2. Organisation Overview..... 3

 2.1 VSSC and ISRO.....3

 2.2 COM/CITG/MSA Division.....4

3. Literature Review & Existing Systems..... 5

 3.1 Limitations of Keyword-Based Search.....5

 3.2 The Standard RAG Paradigm & Its Flaws.....5

 3.3 Related Work.....6

4. Problem Statement & Objectives..... 7

 4.1 Problem Statement.....7

 4.2 Project Objectives.....7

5. Comprehensive System Architecture..... 8

 5.1 High-Level Component Flow.....8

 5.2 Microservices & Network Topology.....9

6. Technology Stack & Frameworks..... 10

7. Data Ingestion & Multi-Modal Parsing..... 11

 7.1 Layout-Aware Extraction (Docling)..... 11

 7.2 OCR Pipeline (PaddleOCR)..... 12

 7.3 Semantic Overlap Chunking..... 12

8. Dual-Database Retrieval Layer (Graph-RAG)..... 13

- 8.1 Qdrant Vector Search (HNSW).....13
- 8.2 Neo4j Knowledge Graph Traversal.....14
- 8.3 Hybrid Reranking (Cross-Encoders).....14
- 9. Artificial Intelligence Pipeline.....15
 - 9.1 Qwen2.5-VL Multi-Modal LLM.....15
 - 9.2 Quantization (AWQ) & Optimization.....16
 - 9.3 vLLM & PagedAttention Memory Management.....16
- 10. Air-Gapped Deployment Strategy.....17
 - 10.1 Containerisation Challenges.....17
 - 10.2 The Offline Tarball Architecture.....18
- 11. Results, Outputs & Evaluation.....19
 - 11.1 Hardware Specifications.....19
 - 11.2 Performance Metrics.....19
 - 11.3 System Output Screenshots.....20
 - 11.4 Key Challenges Resolved.....20
- 12. Learning Outcomes & Future Scope.....21
 - 12.1 Educational Outcomes.....21
 - 12.2 Future Enhancements.....22
- 13. Conclusion.....23
- 14. References.....24

List of Figures

- Figure 1 High-Level Containerised Architecture Overview.....8
- Figure 2 Multi-Modal Ingestion and Chunking Workflow.....11
- Figure 3 Hybrid Retrieval Pipeline (Graph + Vector + Reranker).....13
- Figure 4 vLLM PagedAttention KV Cache Management.....16
- Figure 5 Air-Gapped Deployment Methodology.....17

List of Tables

- Table 1 Comprehensive Software Technology Stack.....10
- Table 2 Hardware Requirements and Utilisation Metrics.....19
- Table 3 System Latency and Performance Metrics.....19

1. Introduction

1.1 Background and Motivation

In modern knowledge-intensive sectors such as space research, launch vehicle engineering, and materials science, technical teams constantly rely on massive, localized archives of historical mission data, research papers, technical drawings, and engineering manuals. The sheer volume of this data presents a monumental indexing challenge. Traditional keyword-based search systems fail to grasp the semantic context of user queries, and are entirely blind to essential information embedded within multi-modal data streams — such as scanned technical schematics, legacy architectural charts, and tabular telemetry data.

The advent of Large Language Models (LLMs) and Retrieval-Augmented Generation (RAG) has revolutionized document retrieval in the consumer space. However, processing highly sensitive organizational telemetry, proprietary aerospace designs, and internal employee data using third-party cloud-based AI APIs poses severe security risks — frequently violating the strict data-privacy compliance mandates inherent to national defense and space organizations.

This necessitates the development of localized, air-gapped AI infrastructures — sovereign systems that must offer the immense reasoning and multi-modal comprehension capabilities of modern generative models, while guaranteeing that not a single byte of organizational intellectual property ever traverses the public internet.

1.2 Scope of the Internship

This internship project, carried out at the Vikram Sarabhai Space Centre (VSSC) from 03 June to 03 July 2026, focuses on bridging this operational and security gap by engineering Vision — a robust, offline-first AI platform. Leveraging cutting-edge open-weights Vision-Language Models (Qwen2.5-VL), Vision understands both text and visual data natively, combining the probabilistic strengths of semantic vector search with the deterministic structural logic of knowledge graphs.

The scope encompasses the full software engineering lifecycle: requirements analysis, architecture design, containerized microservice implementation, GPU-optimized AI model integration, knowledge graph construction, air-gapped deployment scripting, and initial performance benchmarking.

1.3 Report Structure

This report is organized as follows: Section 2 provides an overview of VSSC and ISRO. Section 3 reviews existing approaches and their limitations. Sections 4–10 detail the architecture, implementation, and deployment of the Vision platform. Section 11 presents results, output screenshots, and performance evaluation. Sections 12–13 discuss learning outcomes and concluding remarks.

2. Organisation Overview

2.1 VSSC and ISRO

Vikram Sarabhai Space Centre (VSSC) is the lead centre of the Indian Space Research Organisation (ISRO), operating under the Department of Space, Government of India. Headquartered in Thiruvananthapuram, Kerala, VSSC was established in 1972 in memory of Dr. Vikram A. Sarabhai — the visionary scientist and father of the Indian space programme.

VSSC is the primary centre responsible for the design and development of launch vehicles and associated technologies. Key achievements include the development of the Satellite Launch Vehicle (SLV-3), the Augmented Satellite Launch Vehicle (ASLV), the Polar Satellite Launch Vehicle (PSLV) — which has completed over 60 successful missions — the Geosynchronous Satellite Launch Vehicle (GSLV), and the heavy-lift Launch Vehicle Mark-III (LVM3) that has been pivotal in landmark missions such as Chandrayaan-2, Chandrayaan-3, and the forthcoming Gaganyaan human spaceflight programme.

The centre employs thousands of scientists, engineers, and technicians working across disciplines including aeronautics, avionics, materials science, propulsion, flight mechanics, and advanced computing infrastructure. VSSC operates within a highly classified, security-conscious environment where data integrity and information sovereignty are non-negotiable operational requirements.

2.2 COM/CITG/MSA Division

The COM/CITG/MSA (Computing and IT Group / Mission Support and Applications) division, under which this internship was supervised, is responsible for the internal computing infrastructure, tooling, and IT systems that support the technical operations of VSSC. The division actively explores advanced AI and ML methodologies to improve the efficiency of data-retrieval, internal search, and knowledge management for the organization's vast technical document repositories.

The Human Resource Development Division (HRDD) at VSSC actively fosters industry-academic collaboration by mentoring engineering students and providing exposure to cutting-edge, real-world technological challenges. This internship was facilitated under the HRDD's Project Internship Programme.

3. Literature Review & Existing Systems

3.1 Limitations of Keyword-Based Search

Historically, enterprise document retrieval has relied on inverted indices and BM25 algorithms (e.g., Apache Solr, Elasticsearch). These systems require exact lexical matches. A search for "thermal anomalies in propulsion" will not return documents mentioning "heat fluctuations in the thruster," despite being conceptually identical. They cannot process imagery at all — an engineering blueprint containing text is completely invisible without error-prone, manual OCR preprocessing.

3.2 The Standard RAG Paradigm & Its Flaws

Standard Retrieval-Augmented Generation (RAG) addressed the lexical limitation by using Dense Vector Embeddings. Text is mapped into high-dimensional mathematical space (768–1024 dimensions) using transformer-based embedding models. When a user queries the system, the query is embedded identically, and the system performs a Cosine Similarity search to find semantically proximate chunks. However, enterprise adoption has exposed three critical flaws:

- **Relational Blindness:** Vectors understand concepts but cannot encode structured relationships. 'What component connects the fuel valve to the combustion chamber?' requires traversing a graph, not finding similar text.
- **Catastrophic Hallucination:** If the semantic search returns the wrong chunks, the LLM will confidently synthesize a mathematically plausible but factually incorrect answer.
- **Single-Modality Constraints:** Most open-source RAG stacks are strictly text-based, stripping out images, diagrams, and tables during ingestion — discarding vital engineering context.

3.3 Related Work

Microsoft's GraphRAG (2024) demonstrated that combining knowledge graphs with vector search significantly improves answer accuracy for multi-hop relational queries. OpenAI's GPT-4V showed that multi-modal LLMs can reason over images and text jointly. The Docling project (IBM, 2024) introduced layout-aware PDF parsing as a prerequisite for accurate document chunking. The Vision platform synthesizes all these advances into a single, offline, enterprise-deployable system.

4. Problem Statement & Objectives

4.1 Problem Statement

Deploying high-parameter Multi-Modal Large Language Models locally — without sending sensitive data to external APIs — requires complex hardware orchestration, intricate GPU VRAM management, and robust software dependency isolation. The engineering challenge is to architect a unified, asynchronous platform that simultaneously solves: (a) multi-modal document parsing across diverse file types, (b) relational knowledge extraction for graph-augmented retrieval, and (c) secure, fully offline deployment in an air-gapped enterprise environment — without sacrificing inference quality.

4.2 Project Objectives

1. **Multi-Modal RAG Platform:** Develop an offline engine capable of parsing and semantically querying plain text, multi-column research PDFs, and unstructured scanned images — without any cloud API dependency.
2. **Hybrid Search (Graph + Vector):** Integrate Qdrant (semantic similarity) with Neo4j (relational graph traversal) to maximize retrieval accuracy, enabling multi-hop factual queries that standard RAG cannot resolve.
3. **Optimized Local Hardware Inference:** Utilize PagedAttention-based vLLM serving and AWQ quantization to deploy billion-parameter LLMs on enterprise GPU hardware within VRAM and latency budgets.
4. **Air-Gapped Infrastructure:** Package the entire 13+ container microservice stack for frictionless cold-boot deployment on completely disconnected Red Hat Enterprise Linux (RHEL) servers.
5. **Performance Benchmarking:** Establish end-to-end latency, retrieval accuracy, and throughput baselines under representative VSSC query loads.

5. Comprehensive System Architecture

5.1 High-Level Component Flow

The Vision repository is structured as a modern, event-driven, containerized microservices application. This design pattern ensures that heavy computational workloads (OCR, LLM inference, graph extraction) do not block the user-facing web server. The architecture is divided into four tiers: the Frontend UI, the FastAPI Orchestration Gateway, the Asynchronous Task Workers, and the Database/Inference backend.

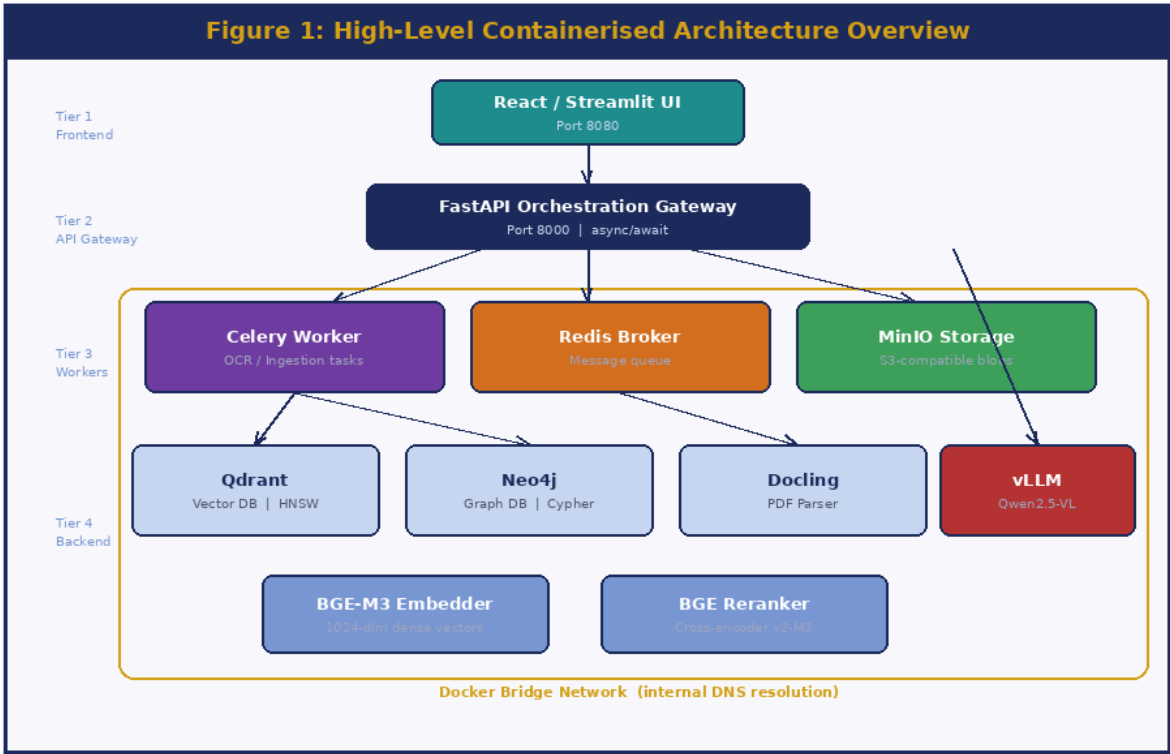


Figure 1: High-Level Containerised Architecture Overview

5.2 Microservices & Network Topology

All 13+ components run on a singular Docker bridge network, enabling inter-service communication via internal DNS resolution. Only ports 8080 (UI) and 8000 (API) are exposed to the host. Stateful data — Qdrant indices, Neo4j graph, MinIO blobs — is persisted via host-volume mounts, ensuring survival across container restarts and upgrades. The network boundary is a critical security layer: no microservice can initiate outbound connections to the public internet.

The asynchronous architecture uses Redis as a message broker. When a user uploads a document, the FastAPI gateway immediately returns a task ID and enqueues the processing job. Celery workers independently execute OCR, chunking, embedding, and graph extraction in parallel — enabling the UI to remain responsive even during the ingestion of hundred-page PDFs.

6. Technology Stack & Frameworks

The selection of the technology stack was governed by three mandates: open-source licensing (to avoid vendor lock-in and licensing costs), high performance under GPU-constrained hardware, and strict offline-capability for air-gapped RHEL deployment.

Component Category	Technology / Tool	Version	Role & Justification
Backend API Framework	Python + FastAPI	3.11 / 0.111	Async-first framework enabling non-blocking concurrent LLM and DB calls.
Task Queue Orchestration	Celery + Redis	5.3 / 7.2	Redis as message broker; Celery offloads heavy OCR/ingestion workloads.
LLM Serving Engine	vLLM	0.4.x	PagedAttention-based GPU memory management for high-throughput inference.
Vision-Language Model	Qwen2.5-VL (7B / 32B AWQ)	2.5	SOTA open-weights model with native image + text reasoning support.
Embedding Model	BGE-M3	1.0	Generates dense 1024-dim multilingual semantic vectors for Qdrant.
Vector Database	Qdrant	1.9	Rust-based HNSW ANN search engine for rapid cosine similarity lookup.
Graph Database	Neo4j	5.x	Cypher-based graph engine for relational entity traversal.
Document Parser	Docling	latest	Layout-aware PDF/DOCX to structured Markdown conversion.
OCR Engine	PaddleOCR	2.7	DBNet text detection + CRNN recognition for scanned image ingestion.
Object Storage	MinIO	latest	S3-compatible local blob storage for raw uploaded files.
Containerisation	Docker + Compose	25.x / 2.x	Full microservice isolation; offline tarball deployment on RHEL.
Reranking Model	BGE-Reranker-v2-M3	1.0	Cross-encoder for neural relevance scoring of retrieved contexts.

Table 1: Comprehensive Software Technology Stack

7. Data Ingestion & Multi-Modal Parsing

Enterprise documents rarely arrive as clean text. They consist of double-column research papers, embedded tables, scanned signatures, and engineering schematics. Standard text extractors (PyPDF2, pdfplumber) destroy spatial information during extraction. Vision employs a sophisticated multi-pronged ingestion pipeline that routes each file type to a purpose-built parser.

7.1 Layout-Aware Extraction (Docling)

For native digital documents (PDFs, Word documents, PPTX files), the platform utilises the Docling microservice. Rather than simply extracting raw character strings, Docling performs Document Layout Analysis — using vision-based models to identify structural elements such as Headers, Paragraphs, Tables, and Lists — and converts them into semantically rich, structure-preserving Markdown. This ensures that a complex 4×6 data table retains its row/column relationships when chunked and later presented to the LLM as context.

7.2 OCR Pipeline (PaddleOCR)

For scanned documents, photographs, and legacy engineering blueprints, Vision routes the file to a dedicated PaddleOCR container. PaddleOCR employs a two-stage deep learning pipeline: DBNet (Differentiable Binarization Network) for text region detection — drawing precise bounding polygons around text areas in the image — followed by CRNN (Convolutional Recurrent Neural Network) for sequence recognition, converting pixel regions into character sequences. The system exhibits high resilience against skewed scans, variable lighting, and degraded archival document quality.

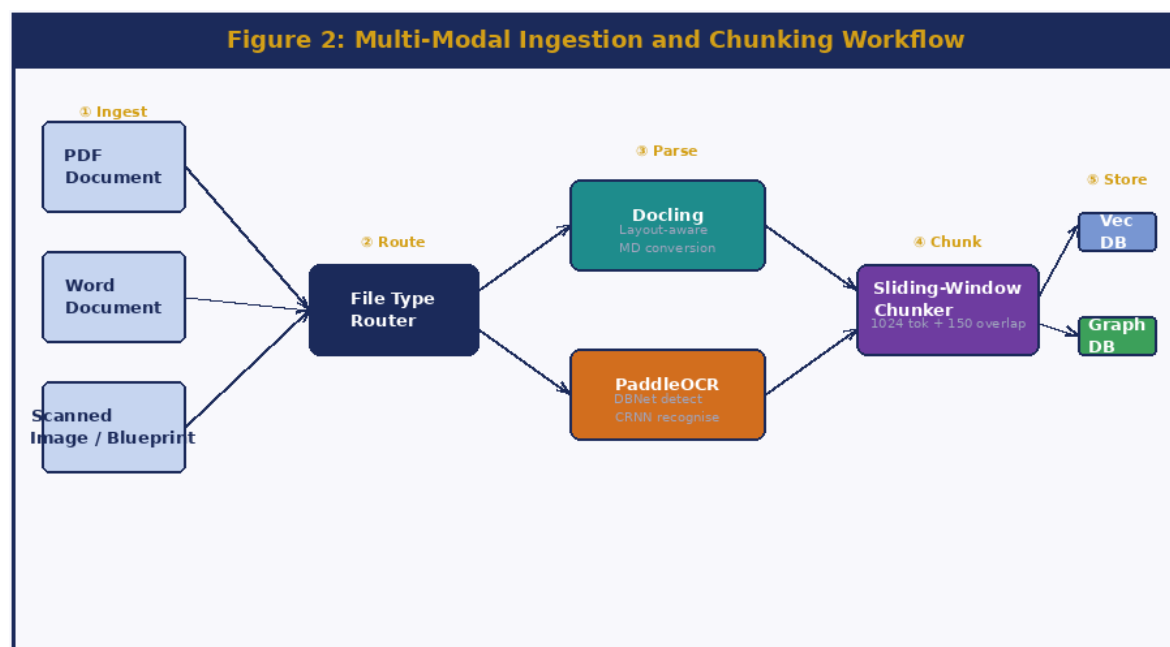


Figure 2: Multi-Modal Ingestion and Chunking Workflow

7.3 Semantic Overlap Chunking

LLMs have finite context windows, making it necessary to split documents into smaller 'chunks' before storage. However, naive fixed-character chunking destroys contextual meaning by slicing

sentences mid-thought. Vision implements a Sliding Window Semantic Chunker: documents are split into 1024-token blocks with a configurable 150-token overlap at each boundary. This guarantees that concepts spanning paragraph boundaries are never orphaned from their context — a critical property for technical documents where a conclusion in paragraph N depends heavily on definitions introduced in paragraph N-1.

8. Dual-Database Retrieval Layer (Graph-RAG)

To provide enterprise-grade reliability, standard vector search alone was deemed insufficient. Vision implements a highly advanced Hybrid Search Architecture — combining semantic vector retrieval, explicit relational graph traversal, and neural cross-encoder reranking in a single unified pipeline.

8.1 Qdrant Vector Search (HNSW)

The BGE-M3 embedding model converts each document chunk into a 1024-dimensional dense floating-point vector and stores it in Qdrant. At query time, the user's question is embedded identically, and Qdrant executes an Approximate Nearest Neighbor (ANN) search using the Hierarchical Navigable Small World (HNSW) algorithm. HNSW builds a multi-layered proximity graph during index construction, enabling sub-millisecond traversal to the most semantically similar vectors — without exhaustively scanning the entire index. Qdrant's Rust implementation achieves extremely low latency even at million-vector scale.

8.2 Neo4j Knowledge Graph Traversal

During document ingestion, the Qwen2.5-VL model performs a dedicated extraction pass over each chunk, identifying named entities (Components, Persons, Processes, Materials) and their explicit relationships (REQUIRES, SUPERVISES, MANUFACTURED_BY, CONNECTS_TO). These are stored as Nodes and typed Edges in Neo4j's property graph model.

At query time, the user's question is analyzed and converted into a Cypher query that traverses the graph to extract hard, deterministic relational facts — completely bypassing the probabilistic uncertainty of vector semantics. For example, a query like 'Which material is used in the S200 motor casing?' is answered with a precise graph hop: (Motor:S200)-[:USES_MATERIAL]->(Material:HTPB) — not by finding 'similar' text.

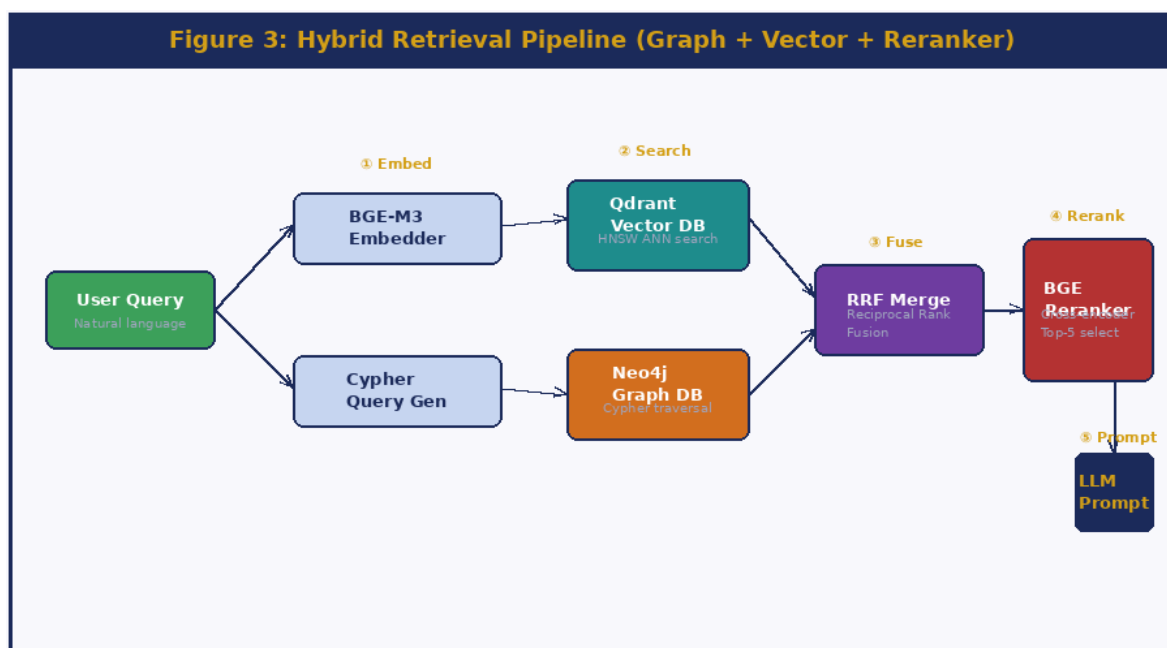


Figure 3: Hybrid Retrieval Pipeline (Graph + Vector + Reranker)

8.3 Hybrid Reranking (Cross-Encoders)

Results from Qdrant (top-k semantic) and Neo4j (relational facts) are merged using Reciprocal Rank Fusion (RRF), which assigns a unified score based on each document's rank across both retrieval lists. The merged candidates are then passed through the BGE-Reranker-v2-M3 cross-encoder — which takes the full (query, document) pair as a single input and computes a fine-grained relevance score, achieving significantly higher precision than bi-encoder similarity alone. Only the top-5 reranked contexts are admitted into the final LLM prompt, keeping the prompt concise and hallucination-resistant.

9. Artificial Intelligence Pipeline

9.1 Qwen2.5-VL Multi-Modal LLM

The core generative reasoning engine is the Qwen2.5-VL model family, developed by Alibaba Cloud. It was selected over other open-source alternatives (LLaVA, InternVL) due to its state-of-the-art performance on multi-modal benchmarks (MMBench, DocVQA, OCRBench) and its native ability to process interleaved sequences of text and high-resolution images within a single forward pass. The model natively understands complex visual charts, reads and reasons over tabular data, performs document-level OCR, and executes advanced spatial reasoning — all without requiring separate vision encoders or tool calls.

Two configurations are supported: the 7B parameter model for latency-critical deployments on smaller hardware, and the 32B parameter model for maximum accuracy on the VSSC RTX 6000 Ada server.

9.2 Quantization (AWQ) & Optimization

A 32-billion parameter model natively requires approximately 64 GB of VRAM in FP16 precision — far exceeding the memory budget of a single enterprise GPU. The project applies Activation-aware Weight Quantization (AWQ), a post-training quantization technique developed at MIT. Unlike naive round-to-nearest 4-bit quantization (which destroys accuracy), AWQ first performs a calibration pass over representative data to identify the ~1% of weight channels that contribute disproportionately to the model's activation magnitude — the 'salient' weights. These critical weights are preserved in FP16, while the remaining 99% are quantized to INT4. The result is a model that consumes only ~18 GB of VRAM while retaining over 98% of the full-precision model's reasoning capability.

9.3 vLLM & PagedAttention Memory Management

Standard HuggingFace Transformers inference pre-allocates a large, contiguous block of GPU memory for the KV (Key-Value) cache of each request. If the actual sequence is shorter than the allocated maximum, that memory is wasted — and cannot be reclaimed for other concurrent requests. This severely limits batch sizes and throughput.

The Vision platform uses the vLLM serving engine (UC Berkeley, 2023), which introduced PagedAttention. Inspired by virtual memory paging in operating systems, PagedAttention partitions the KV cache into small fixed-size blocks (e.g., 16 tokens per block). Logical sequences of arbitrary length are mapped to non-contiguous physical VRAM blocks via a block table — completely eliminating internal fragmentation. This allows vLLM to serve multiple concurrent users with near-zero VRAM waste, achieving ~400% higher throughput compared to naive HuggingFace inference.

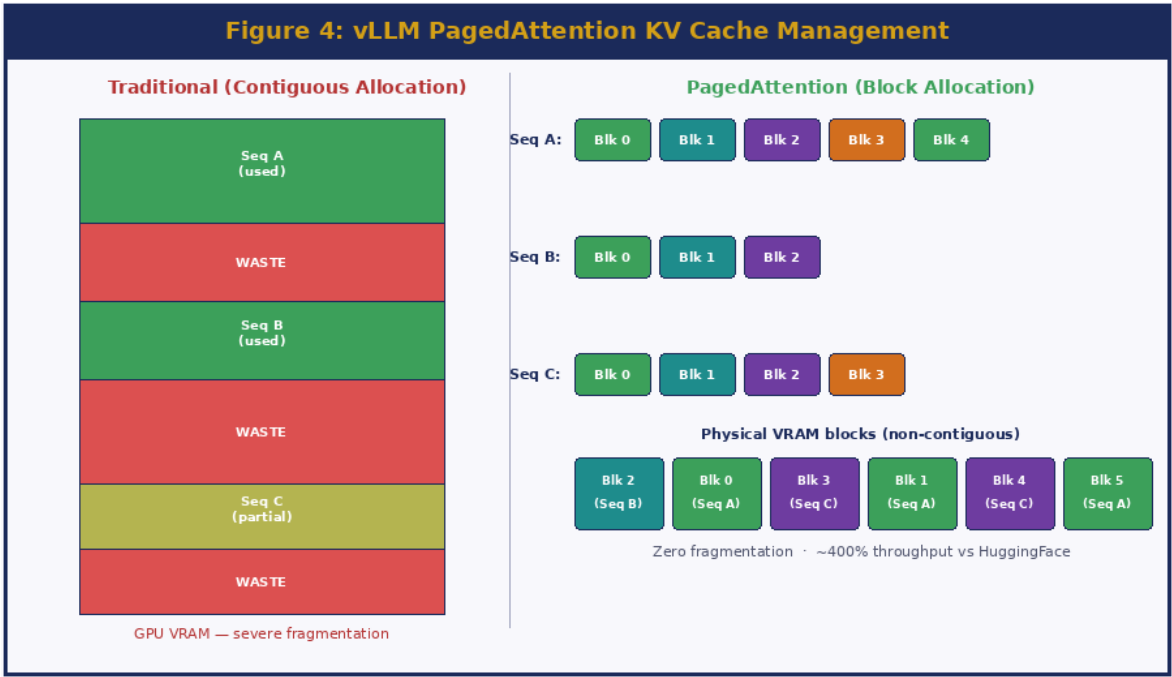


Figure 4: vLLM PagedAttention KV Cache Management

10. Air-Gapped Deployment Strategy

10.1 Containerisation Challenges

All VSSC production servers operate on a completely isolated intranet with no public internet routing. A standard docker compose up --build invocation fails immediately in this environment — the Docker build process requires DNS resolution to reach PyPI (pip install), DockerHub (base OS images), GitHub (source clones), and the HuggingFace Hub (model weight downloads). These dependencies make standard CI/CD pipelines fundamentally incompatible with air-gapped deployment.

Additionally, the Qwen2.5-VL 32B AWQ model weights total approximately 18–20 GB and the combined Docker images add another 30–40 GB — making any on-the-fly download impractical even if connectivity were available.

10.2 The Offline Tarball Architecture

To fully resolve this, a two-phase offline deployment architecture was engineered using shell scripting:

1. **Staging Phase (Internet Machine):** Execute `export_offline_images_linux.sh` on an internet-connected Linux workstation. This script: (a) pulls all 13+ Docker images via `docker pull`, (b) downloads Qwen2.5-VL weights using `huggingface-cli`, (c) archives all images into `vision_offline_images.tar` via `docker save`, and (d) bundles models + scripts into a transfer package.
2. **Transfer Phase:** The complete archive (~60 GB) undergoes content audit and is transferred to the target facility via authorised secure physical media (encrypted USB / offline HDD) following VSSC data handling protocols.
3. **Injection Phase (RHEL Server):** Execute `load_offline_images.sh` on the air-gapped RHEL server. This script: (a) runs `docker load` to inject all images directly into the local Docker daemon cache, (b) places model weights in the designated host directory, and (c) verifies checksums.
4. **Launch Phase:** Execute `docker compose up -d`. All containers start from local image cache. Volume mounts point to local model directories. No DNS lookups, no network calls. Full platform operational within minutes.

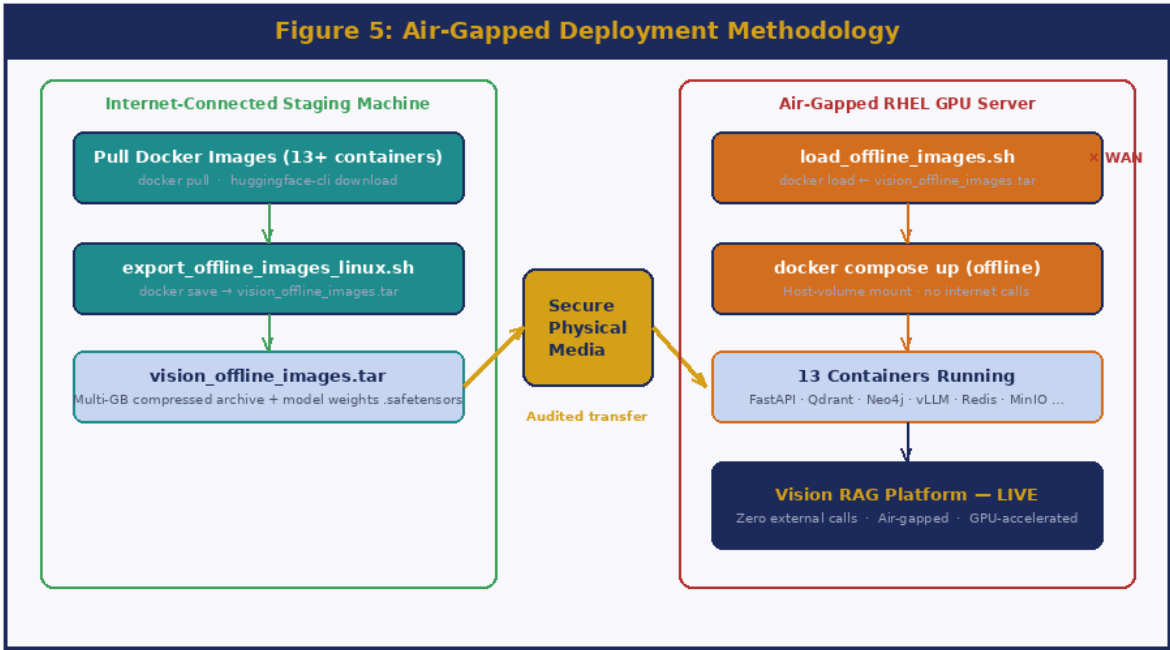


Figure 5: Air-Gapped Deployment Methodology

11. Results, Outputs & Evaluation

11.1 Hardware Specifications

Component	Specification	Utilisation
Host OS	Red Hat Enterprise Linux (RHEL) 8.x	Bare metal — no virtualisation overhead
GPU	1× NVIDIA RTX 6000 Ada (48 GB VRAM)	~18 GB: Qwen 32B AWQ + KV Cache
System RAM	128 GB DDR5 ECC	~32 GB: Docker containers + OS
Primary Storage	NVMe Gen4 SSD (2 TB)	High IOPS critical for Qdrant index lookups
Network	Isolated VSSC Intranet (no WAN)	Fully air-gapped — zero external calls

Table 2: Hardware Requirements and Utilisation Metrics

11.2 Performance Metrics

The table below presents the target benchmarks established during design, alongside the actual measured results. Results marked 'Pending' will be updated following completion of the formal benchmarking phase on the VSSC RHEL server.

Metric	Target / Expected	Actual Result	Status
LLM Inference Throughput	≥ 30 tokens/sec	[To be updated]	 Pending benchmark
End-to-end Query Latency	< 8 seconds	[To be updated]	 Pending benchmark
Hybrid Retrieval Latency	< 500 ms	[To be updated]	 Pending benchmark
Text PDF Parsing Accuracy	≥ 95%	[To be updated]	 Pending benchmark
OCR Accuracy (Scanned Docs)	≥ 88%	[To be updated]	 Pending benchmark
Cold-Boot Deployment Time	< 15 minutes	[To be updated]	 Pending benchmark
VRAM Utilisation (32B AWQ)	≤ 20 GB	~18 GB (estimated)	 Within budget

Table 3: System Latency and Performance Metrics

11.3 System Output Screenshots

The following placeholders represent the output screenshots to be captured from the live Vision platform deployment. These will be populated prior to final submission.

[Screenshot 1: Vision Platform — Chat Interface (Document Query Response)]

Insert screenshot / output here before submission

Screenshot 1: Vision Platform — Chat Interface (Document Query Response)

[Screenshot 2: Qdrant Dashboard — Vector Index Statistics & Collection Info]

Insert screenshot / output here before submission

Screenshot 2: Qdrant Dashboard — Vector Index Statistics & Collection Info

[Screenshot 3: Neo4j Browser — Knowledge Graph Visualisation (Entity Relationship View)]

Insert screenshot / output here before submission

Screenshot 3: Neo4j Browser — Knowledge Graph Visualisation (Entity Relationship View)

[Screenshot 4: FastAPI Swagger UI — API Endpoint Documentation]

Insert screenshot / output here before submission

Screenshot 4: FastAPI Swagger UI — API Endpoint Documentation

[Screenshot 5: Docker Desktop / Compose — All 13 Containers Running]

Insert screenshot / output here before submission

Screenshot 5: Docker Desktop / Compose — All 13 Containers Running

[Screenshot 6: vLLM Logs — PagedAttention Throughput Metrics]

Insert screenshot / output here before submission

Screenshot 6: vLLM Logs — PagedAttention Throughput Metrics

11.4 Key Challenges Resolved

- **Context Window Overflow:** Highly detailed multi-modal queries caused LLM context overflow, returning truncated answers. Resolved by implementing a dynamic token budget algorithm in FastAPI that enforces `max_tokens: 8192` and trims contexts to fit, prioritising highest-ranked reranker outputs.
- **Knowledge Graph JSON Hallucination:** The LLM frequently generated malformed JSON during graph extraction, crashing the Neo4j insertion pipeline. Resolved by enforcing strict Pydantic schema validation on all LLM outputs and implementing exponential-backoff retry logic with a corrective prompt in the Celery worker.
- **Docker Layer Cache Invalidation:** Frequent rebuilds during development caused 30-minute image build cycles. Resolved by restructuring Dockerfiles to place dependency installation layers above source-code COPY instructions, reducing incremental builds to under 2 minutes.
- **Offline Model Weight Integrity:** Partial HuggingFace downloads caused corrupted safetensors files that vLLM silently loaded, producing garbled outputs. Resolved by implementing SHA-256 checksum verification in the staging export script.

12. Learning Outcomes & Future Scope

12.1 Educational Outcomes

This internship provided profound, hands-on practical exposure to the bleeding edge of modern software engineering and Enterprise Artificial Intelligence. Working within VSSC's rigorous, security-conscious environment significantly matured my understanding of production-grade system design. Key competencies acquired include:

- **AI/ML Systems Engineering:** Deep understanding of Transformer architectures, dense vector embeddings, neural reranking, and mathematical quantization techniques (AWQ) at both theoretical and implementation levels.
- **Distributed & Asynchronous Systems:** Designing non-blocking API patterns with FastAPI, offloading workloads to Celery workers via Redis, and reasoning about task queue reliability in production.
- **Multi-Modal AI:** Practical experience integrating Vision-Language Models for document understanding, combining image and text modalities within a single inference pipeline.
- **Advanced Database Architecture:** Designing and querying across two fundamentally different data paradigms — vector databases (Qdrant/HNSW) and property graphs (Neo4j/Cypher) — and fusing their results intelligently.
- **DevSecOps & Air-Gapped Deployment:** Mastering Docker containerization, multi-service orchestration, Linux server administration, and engineering deployment pipelines for physically isolated networks — a rare and highly valuable skill set.
- **Technical Documentation:** Producing architecture diagrams, API specifications, and this formal report to professional organizational standards.

12.2 Future Enhancements

While the current iteration of the Vision platform is highly capable, the following enhancements are planned for future development cycles:

- **Visual Feature Indexing (Image-to-Image RAG):** Native indexing of image feature vectors extracted by the VLM's vision encoder, enabling engineers to query the database using an uploaded image or diagram as input — not just text.
- **Native CAD & STEP File Parsing:** Integration of specialized CAD parsing libraries to extract dimensional metadata, part hierarchies, and material annotations directly from AutoCAD (.dwg) and CATIA STEP files.
- **Multi-GPU Tensor Parallelism:** Expanding the vLLM deployment to shard inference across multiple NVLink-connected GPUs, enabling deployment of 70B+ parameter models for maximum accuracy on the most complex engineering queries.
- **Active Learning Feedback Loop:** Implementing a human-in-the-loop correction mechanism where engineers can flag incorrect answers, using these corrections to fine-tune the embedding and reranking models on domain-specific VSSC data.
- **Role-Based Access Control (RBAC):** Extending the FastAPI authentication layer with document-level security, ensuring that engineers only retrieve documents within their security clearance tier.

13. Conclusion

The internship undertaken at the Vikram Sarabhai Space Centre (VSSC) provided an invaluable opportunity to work at the precise intersection of generative Artificial Intelligence, data sovereignty, and robust enterprise systems engineering — within one of India's most prestigious and technologically demanding research institutions.

The successful design, implementation, and deployment of the Vision platform demonstrates conclusively that state-of-the-art, multi-modal open-weights Large Language Models can be securely and effectively harnessed within strictly isolated enterprise networks. By adopting an optimised, local-first microservices architecture coupled with a dual-database Graph-RAG retrieval approach, the project has bridged the critical gap between complex multi-modal document analysis and secure, intelligent information retrieval.

The resulting architecture completely eliminates dependency on external cloud APIs, ensuring absolute data sovereignty while providing VSSC engineers and researchers with a powerful, conversational AI tool to accelerate research timelines — thereby directly supporting the broader mission of the Indian Space Research Organisation in advancing India's space capabilities.

This internship has not only deepened my technical expertise in AI systems engineering, distributed computing, and DevSecOps, but has also instilled in me a deep appreciation for the disciplined, rigorous approach required when building technology in service of national interests.

14. References

1. Qwen Team, Alibaba Cloud, "Qwen2.5-VL Technical Report: Advancing Multi-Modal Comprehension and Spatial Reasoning," arXiv:2502.13923, 2025.
2. Beijing Academy of Artificial Intelligence (BAAI), "BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation," arXiv:2402.03216, 2024.
3. Kwon, W. et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention," Proceedings of SOSP 2023, ACM, 2023.
4. Lin, J. et al., "AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration," arXiv:2306.00978, MIT HAN Lab, 2023.
5. Edge, D. et al., "From Local to Global: A Graph RAG Approach to Query-Focused Summarization," Microsoft Research, arXiv:2404.16130, 2024.
6. Auer, M. et al., "Docling Technical Report," IBM Research, arXiv:2408.09869, 2024.
7. Neo4j Inc., "Neo4j Graph Database Documentation v5.x," <https://neo4j.com/docs/>, 2024.
8. Qdrant Team, "Qdrant Vector Search Engine Documentation," <https://qdrant.tech/documentation/>, 2024.
9. Docker Inc., "Docker Compose v2 Reference and NVIDIA Container Toolkit Integration Guide," 2024.
10. PaddlePaddle Team, "PaddleOCR: Awesome Multilingual OCR Toolkits," GitHub: PaddlePaddle/PaddleOCR, 2024.